Experiment- 8

Branch: MCA (AI&ML)	**Semester: 2**
Student Name: Ajay Rakwal	**UID:** 25MCI10329
Subject Name: Technical Training	**Subject Code**: 25CAP-652
Section/Group: MAM-1(A)	**Date of Performance:** 21-04-2026

Aim of the program:

To understand and apply transaction control in PostgreSQL using BEGIN, COMMIT, ROLLBACK and SAVEPOINT to maintain data integrity during database operations.

Software Requirements

- **Operating System:** Windows / Linux
- **Database Management System:** MySQL / Oracle / PostgreSQL
- **SQL Interface:** MySQL Workbench /Web Based / pgAdmin

Objective

Apply transaction control commands for safe and reliable database updates.

Procedure of the Practical

- Open PostgreSQL (pgAdmin) and connect to the required database.
- Create the Payroll table with constraints such as primary key and salary check condition (> 0).
- Insert sample records into the Payroll table for testing transaction operations.

- Start a transaction using BEGIN and perform update operations, including an invalid update.
- Use ROLLBACK to undo all changes made in the transaction.
- Verify the table data using a SELECT query to confirm rollback execution.
- Start a new transaction using BEGIN and perform multiple update operations.
- Create a SAVEPOINT to mark a specific state within the transaction.
- Perform further updates, including an invalid operation.
- Use ROLLBACK TO SAVEPOINT to undo changes after the savepoint.
- Use COMMIT to permanently save valid changes in the database.
- Verify final changes using a SELECT query.

Practical / Experiment Steps

```
Create table payroll(

-- CREATE TABLE

CREATE TABLE payroll(

emp_id int primary key,

emp_name varchar(50),

salary decimal(10,2) check(salary>0)

);

-- INSERT RECORDS

Insert into payroll values

(1,'Ajay',30000),

(2,'Purnima',40000),

(3,'Neeraj',50000);

-- START TRANSACTION

begin;
```

```
-- INVALID UPDATE (NEGATIVE SALARY)
update payroll
set salary=-1000
where emp_id=3;
-- VALID UPDATE
update payroll
set salary=1000
where emp_id=3;
-- ROLLBACK ALL CHANGES
rollback;
-- DISPLAY DATA
select * from payroll;
-- START NEW TRANSACTION
begin;
-- UPDATE SALARY EMPLOYEE 1
update payroll
set salary=salary+5000
where emp_id=1;
-- CREATE SAVEPOINT
savepoint sp1;
-- UPDATE SALARY EMPLOYEE 2
update payroll
set salary=salary+7000
where emp_id=2;
```

```
-- INVALID UPDATE

update payroll

set salary=-1000

where emp_id=3;

-- ROLLBACK TO SAVEPOINT

rollback to sp1;

-- COMMIT CHANGES

commit;
```

I/O Analysis (Input / Output)

Input

- Table creation query with constraints
- Sample data insertion commands
- Transaction control statements (BEGIN, COMMIT, ROLLBACK, SAVEPOINT)
- UPDATE queries including valid and invalid operations
- SELECT query to verify results

Output

- Payroll table created successfully
- Records inserted correctly
- Invalid update (negative salary) rejected due to constraint
- ROLLBACK restored original data after first transaction
- SAVEPOINT created and partial rollback executed successfully
- Valid updates committed and reflected in final table data
- Data integrity maintained throughout transaction operations

OUTPUT:

Table Created:

```
Data Output   Messages   Notifications
CREATE TABLE

Query returned successfully in 70 msec.
```

INSERT VALUES:

```
Data Output   Messages   Notifications
INSERT 0 3

Query returned successfully in 124 msec.
```

UPDATE:

```
Data Output   Messages   Notifications
UPDATE 1

Query returned successfully in 139 msec.
```

SAVEPOINT:

```
Data Output   Messages   Notifications
SAVEPOINT

Query returned successfully in 127 msec.
```

ROLLBACK :

```
Data Output   Messages   Notifications
ROLLBACK

Query returned successfully in 134 msec.
```

Learning Outcome:

- Understand the concept of transaction control in PostgreSQL.
- Learn how to use BEGIN, COMMIT, and ROLLBACK to manage database transactions.
- Gain practical experience with SAVEPOINT for partial rollback operations.
- Understand how transaction control helps maintain data integrity and consistency.
- Develop the ability to handle errors and recover from invalid operations in database systems.